University of Florida

Department of Electrical and Computer Engineering

Assignment on Abstractions, Dynamic Power, Amdahl's Law, and gem5 Setup

Name : Shahriar Ahmed
Course Code : EEL 5764
Course : Computer Architecture
Submitted to : Professor Dr. Sandip Ray

Part - A

Problem - 1 : (Latency vs Throughput Dimensions)

Part - A

We know that cpu execution time

$$CPU_{time} = \text{Instruction count} \times CPI \times \text{clock cycle time}$$

Doubling the frequency and keeping everything same,

$$CPU_{time} \propto \text{clock cycle time} \propto \frac{1}{\text{frequency}}$$

$$\frac{CPU_{time_new}}{CPU_{time_old}} = \frac{\text{frequency_old}}{f_new}$$

$$= \frac{2}{4} = 0.5$$

$$\therefore CPU_{time_new} = 0.5 \times CPU_{time_old}$$

∴ It takes half the time to execute a single program compared to before.

$\therefore$ Latency is halved

(2) Throughput = number of instructions per unit time

$$= IPC \times f$$

$$= \frac{f}{CPI}$$

As CPI is constant,

$$\text{Throughput} \propto f$$

$$\therefore \quad \frac{\text{Throughput}_{new}}{\text{Throughput}_{old}} = \frac{f_{new}}{f_{old}} = \frac{4}{2} = 2$$

$\therefore$ Instruction throughput doubles

Part-B

(1) A single thread executes on one core at a time. The newly added four cores do not IC, CPI & f. Therefore, execution time of one core is unchanged

and so, latency of a single threaded database look-up task is unchanged.

(2) Assuming ideal operation, (highly parallel) web servers will delegate concurrent work to multiple cores.

Overall,

$$\frac{\text{throughput}_{new}}{\text{throughput}_{old}} = \frac{1+4}{1} = 5$$

Throughput will be 5 times compared to before.

Real life scenerio will be heavily bounded by different factors.

Part-C

Design optimization is mainly focused on workload's demand. The thumb rule is

(i) Optimize for latency:

A computer system with high priority deadline must prioritize latency. Worst case single task latency is the only metric that matters. A missed deadline is a safety issue.

for example, a car's brake system must function (braking commands) within a short time.

(ii) Optimize for throughput:

A system handling many independent tasks should prioritize throughput. Usually cost function is work completed per unit time. for example, a web server or warehouse AI training cluster benefits from multiple cores & sufficient memory bandwidth.

Problem-2 : CMOS Dynamic Power Scaling & the power wall

Part-A

We know that

$$P_{dynamic} = \alpha \times C \times V^2 \times f$$

Assuming activity factor (α) is constant,

$$C_{new} = (1 - 0.3) \times C_{old} = 0.7 \times C_{old}$$

$$f_{new} = \frac{4}{3.5} \times f_{old}$$

$$= 1.1428 \times f_{old}$$

$$\approx 1.143 \times f_{old}$$

$$\frac{V_{new}}{V_{old}} = \frac{0.95}{1.2} = 0.791667$$

$$\Rightarrow \left(\frac{V_{new}}{V_{old}}\right)^2 = 0.791667^2 = 0.6267$$

$$\therefore \quad \frac{P_{new}}{P_{old}} = \frac{C_{new} \times V_{new}^2 \times f_{new}}{C_{old} \times V_{old}^2 \times f_{old}}$$

$$= \left(\frac{C_{new}}{C_{old}}\right) \times \left(\frac{V_{new}}{V_{old}}\right)^2 \times \frac{f_{new}}{f_{old}}$$

$$= 0.7 \times 0.6267 \times 1.142$$

$$= 0.501389$$

$$\therefore P_{new} = 0.501389 \times P_{old}$$

$$= 0.501389 \times 25 \text{ W}$$

$$= 12.5347 \text{ W}$$

Part - B

$$\frac{f_{boost}}{f_{old}} = \frac{4.8}{3.5} = 1.3714$$

$$\left(\frac{V_{boost}}{V_{old}}\right)^2 = \left(\frac{1.15}{1.20}\right)^2 = 0.918403$$

$$\frac{P_{boost}}{P_{old}} = 0.7 \times 0.918403 \times 1.371429$$

$$= 0.881667$$

$$\therefore P_{boost} = 0.881667 \times 25 \text{ W}$$

$$= 22.0417 \text{ W}$$

Part-C

	P (W)	V (V)	f (GHz)
old	25	1.2	3.5
new (A)	12.53	0.95	4
boosted (B)	22.04	1.15	4.8

From the equation

$$P \propto C \times V^2 \times f$$

Power is proportional to frequency but squarely proportional to voltage

Just like that, moving from Part A to Part B

we increased frequency 20% but ended up almost doubling up our power.

$$\frac{P_{boost}}{P_{new}} = \frac{f_{boost}}{f_{new}} \times \left(\frac{V_{boost}}{V_{new}}\right)^2$$

$$= 1.2 \times 1.47$$

Power scaling largely comes from voltage scaling (1.47, bigger factor), only 1.2 voltage factor contributed.

Supply voltage can not decrease indefinitely while maintaining the required switching speed (capacitor charge-discharge). Lowering threshold voltage also increases leakage. So, 1.0V is the practical scaling limit.

Once voltage scaling is fixed, raising frequency increases power consumption. Eventually, cooling dictates the prevention of frequency rise. This is the physical origin of power wall.

Problem-3 : Speedup Limits & Amdahl's law

Part-A

We know that

$$\text{overall speedup} = \frac{1}{(1-f) + \frac{f}{s}}$$

unenhanced fraction. $= 1 - f = 1 - 0.65 = 0.35$

$$\therefore \text{overall speedup} = \frac{1}{(1-f) + \frac{f}{s}}$$

$$= \frac{1}{0.35 + \frac{0.65}{40}}$$

overall speedup = 2.73

Part B

As local speedup factor approaches infinity,

$$\lim_{s \to \infty} \text{overall speedup} = \lim_{s \to \infty} \frac{1}{(1-f) + \frac{f}{s}}$$

$$= \frac{1}{(1-f) + \lim_{s \to \infty} f/s}$$

$$= \frac{1}{1-f}$$

$$\therefore \text{speedup limit} = \frac{1}{1-f} = \frac{1}{1-0.65}$$

$$= 2.857$$

∴ Maximum theoretical speedup limit is 2.857 times and it is limited by the unenhanced portion $(1-f)$ of the system.

Problem-3: Part-C

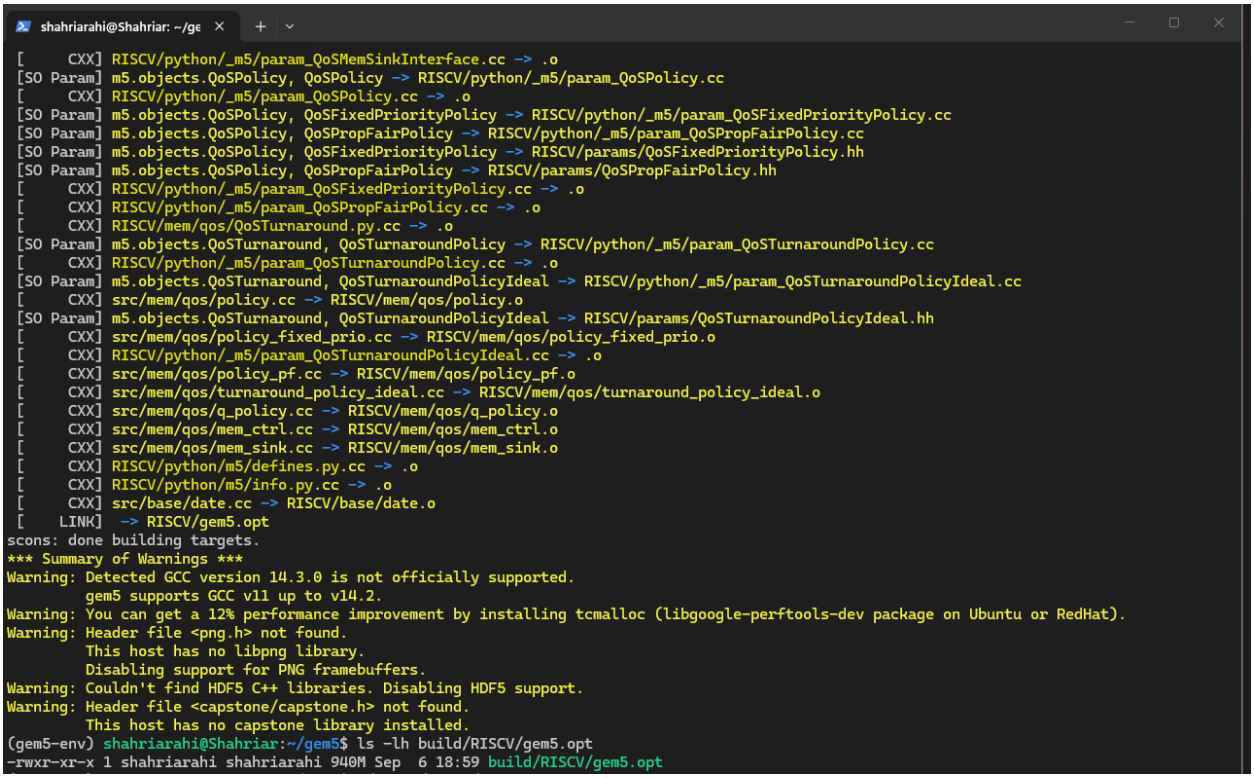
The bottleneck is that 35% of the baseline execution time cannot be accelerated by the NPU. This could include data transfers, CPU control instructions, synchronization, or non-matrix computations. Even if NPU's operations take zero time, the application must still complete the unenhanced portion's work in the normal way. Therefore, acceleration alone cannot exceed the speedup ceiling.

This explains why architects prioritize **"making the common case fast"**: an optimization's value depends on how much of the application's total execution time it saves. A spectacular speedup of a rarely used operation can have very little effect on the whole program. For example, accelerating an operation that occupies only 1% of runtime yields only about a 1% overall performance improvement. A complex hardware unit may therefore consume substantial area, power, and design effort while delivering little practical benefit.

Part-B : Hands-On gem5 Simulation Exercise

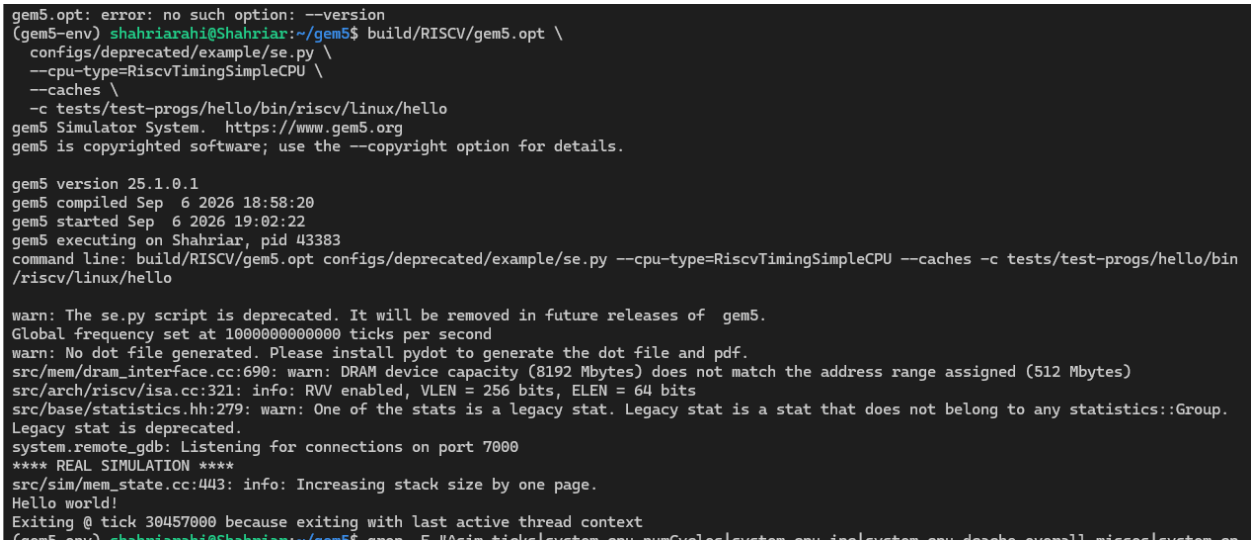
Step-1: SCons Simulator Compilation

The SCon build has been conducted successfully.



Step-2: Simple Executable Run in SE Mode

I have used the commands from the assignment and generated a “Hello World” line as shown below.



Step-3: Navigating Telemetry & Metric Extraction

Gem5 Stat Key	Physical Hardware Metric	Extracted Value
Sim_ticks	Total simulated benchmark runtime in picoseconds (ps). (Include your conversion to nanoseconds (ns)).	30,457,000
System.cpu.numCycles	Total simulated processor clock cycles spent processing the dynamic instruction stream.	60,914
System.cpu.ipc	Instructions executed per clock cycle. (Compute its reciprocal to find the effective Cycles Per Instruction, CPI).	0.090587

System.cpu.dcache.overall_misses	Total cache lookup misses recorded on the L1 Data Cache interface during execution.	135 misses
System.cpu.dcache.overall_miss_rate	The fraction of L1 Data Cache references that resulted in misses, causing memory hierarchy stalls.	0.065217

Step-4: Reflection

Hello World took 60,914 cycles to retire its instruction stream, an IPC of 0.0906 (≈11 CPI). For a workload this small, that figure says more about startup cost than steady-state behavior: the instruction count is dominated by process setup and a single write syscall, so the pipeline has little opportunity to fill. The 135 L1 data cache misses (6.52%) are consistent with this — cold-start compulsory misses on a working set that is never revisited.